



КУРС «ОБЪЕКТНО-ОРИЕНТИРОВАННОЕ ПРОГРАММИРОВАНИЕ» · 1 СЕМЕСТР

08

Лекция 8
Наследование

Одиночное, множественное и виртуальное · модификаторы доступа · порядок конструирования · наследование конструкторов · name hiding · срезка · композиция · final · ромб · vbptr · примеси и интерфейсы

C++20 | Наследование | 12 тем

Вспоминаем, прошлую лекцию

```
8   struct Array {
9       size_t _size{0};
10      Payload * _array{nullptr};
11
12      Array() {};
13      bool resize(size_t new_size) {
14          if (new_size <= _size) return false;
15
16          Payload * _new_array = new Payload[new_size];
17          if (_array) {
18              for(size_t i=0;i<_size;++i)
19                  _new_array[i] = _array[i];
20              delete _array;
21          }
22          _array = _new_array;
23          _size  = new_size;
24          return true;
25      }
26      ~Array() {
27          delete [] _array;
28      }
29  };
```



ПЛАН ЛЕКЦИИ

Двенадцать тем сегодняшней лекции

01 Концепция и синтаксис

02 Модификаторы наследования

03 Порядок конструирования

04 Наследование конструкторов

05 Сокрытие имён

06 Срезка объектов

> **Сквозная идея:** наследование — это заявление о типе, а не способ переиспользовать код. Всё остальное — модификаторы, порядок конструкторов, срезка, ромб — следствия этого контракта.

07 Приватное наследование vs композиция

08 final и «предпочитайте композицию»

09 Множественное наследование

10 Проблема ромба

11 Виртуальное наследование

12 Примеси, интерфейсы, альтернативы

Наследование — это отношение is-a

```
class Animal {
protected:
        std::string name_;
public:
        Animal(std::string n) : name_(std::move(n)) {}
        void info() const { std::cout << name_ << '\n'; }
};

class Dog : public Animal {std::string
        breed_;
public:
        Dog(std::string n, std::string b)
                : Animal(std::move(n)), breed_(std::move(b)) {}

        void bark() const { std::cout << name_ << ": Гав!\n"; }
};

Animal& a = rex;              // Dog можно использовать как Animal
```

- **public-наследование** сохраняет подстановочность: Dog везде, где ждут Animal
- **protected**-члены видны наследнику, **private** — нет
- база инициализируется в **списке инициализации** конструктора
- общие поля и поведение не дублируются

Проверка

Если наследник не может вести себя как база — это не наследование, а переиспользование кода.

ПРИМЕР

Смотрим пример: 01_inheritance_basics

Разбираем после теории

Концепция наследования и синтаксис

Открыть папку примера →

- Базовый Animalи производный Dog
- Доступ к protected-членам базы
- Инициализация базы в списке инициализации
- Подстановка Dogвместо Animalпо ссылке

Код лежит в examples/lecture_08_inheritance/01_inheritance_basics, сборка — через CMake.

Модификатор меняет доступ к наследию

```
class Base {
public:    int pub   = 1; protected:
int prot = 2; private:    int priv = 3;
};


class Pub       : public Base               {}; // pub→public class Prot :
protected Base {}; // pub→protected class Priv : private Base
                   {}; // pub→private

// priv недоступен наследнику
// ни при каком модификаторе
```

Член	public	protected	private
public	public	protected	private
protected	protected	protected	private
private	недоступен		

Для `class`наследование по умолчанию **private**, для `struct`— `public`. Для is-a нужен явный `public`.

ПРИМЕР

Смотрим пример: 02_access_modifiers

Разбираем после теории

Модификаторы наследования и таблица доступа

Открыть папку примера →

- Три наследника одного Base
- Что доступно внутри и снаружи
- Доступ к privateтолько через методы базы

Код лежит в examples/lecture_08_inheritance/02_access_modifiers.

База живёт дольше наследника

```
class Member {std::string tag_;
public:
        explicit Member(std::string t) : tag_(std::move(t)) {std::cout << "    Member("
                << tag_ << ")\n";
        }
        ~Member() { std::cout << "                    ~Member(" << tag_ << ")\n"; }
};

class Base {
        Member m_{"Base::m"};
public:
        Base()        { std::cout << "Base()\n"; }
        ~Base() { std::cout << "~Base()\n"; }
};

class Derived : public Base {Member
        m_{"Derived::m"};
public:
        Derived()          { std::cout << "Derived()\n"; }
        ~Derived() { std::cout << "~Derived()\n"; }
};
```

- **создание:** база → члены базы → члены наследника → тело
- **разрушение:** строго наоборот
- порядок задают **список баз** и **объявление полей**, а не список инициализации
- база в конструкторе наследника уже готова

```
    Member(Base::m)
Base()
    Member(Derived::m)
Derived()
~Derived()
    ~Member(Derived::m)
~Base()
    ~Member(Base::m)
```

ПРИМЕР

Смотрим пример: 03_construction_order

Разбираем после теории

Порядок конструирования и разрушения

Открыть папку примера →

- Печать каждого шага конструктора и деструктора
- Члены базы и наследника — тоже участники порядка
- Сверьте вывод с ожидаемой трассой

Код лежит в examples/lecture_08_inheritance/03_construction_order.

using Base::Base — и все конструкторы ваши

```
class Base {
public:
        explicit Base(int x) { std::cout << "Base(int)\n"; }Base(int x, double y) { std::cout <<
        "Base(int,double)\n"; }
};

class Derived : public Base {std::string
        label_{"<нет>"};                                // default member init
public:
        using Base::Base;                // наследуем оба конструктора

        Derived() : Base(0), label_{"по умолчанию"} {std::cout <<
                "Derived()\n";
        }
};

Derived d1{42}; Derived d2{1,    // Base(int)
3.14}; Derived d3;               // Base(int,double)
                                 // Derived()
```

- наследуются **все** конструкторы базы одной строкой
- поля наследника унаследованный конструктор **не** инициализирует
- для них работают **default member initializers**
- копирующий и перемещающий конструкторы так не наследуются

Зачем

Меньше дублирования: добавили конструктор в базу — наследник получил его автоматически.

ПРИМЕР

Смотрим пример: 04_inheriting_constructors

Разбираем после теории

Наследование конструкторов

Открыть папку примера →

- Два унаследованных конструктора и один собственный
- Поле наследника остаётся с default-значением
- Копирование — отдельный, не наследуемый конструктор

Код лежит в examples/lecture_08_inheritance/04_inheriting_constructors.

Одно имя в наследнике скрывает все перегрузки

```
class Base {
public:
        void foo(int x)                 { std::cout << "Base::foo(int)\n"; }
        void foo(double x) { std::cout << "Base::foo(double)\n"; }
};

class Hiding : public Base {
public:
        void foo(int x) { std::cout << "Hiding::foo(int)\n"; }
        // Base::foo(double) СКРЫТА
};

class Using : public Base {
public:
        using Base::foo;                // воскрешаем перегрузки базы
        void foo(int x) { std::cout << "Using::foo(int)\n"; }
};

Hiding h;          const double pi = 3.14;
h.foo(pi);                         // Hiding::foo(int)! 3.14 -> 3
h.Base::foo(pi);                   // обход: явная квалификация
```

- поиск имени останавливается на наследнике
- это не перегрузка и не переопределение, а **сокрытие**
- using Base::fooобъединяет наборы перегрузок без using
- вызов молча выберет не ту функцию

> Правило: добавили метод с существующим именем — почти всегда нужен using Base::имя;.

ПРИМЕР

Смотрим пример: 05_name_hiding

Разбираем после теории

Сокрытие имён и директива using

Открыть папку примера →

- Два наследника: с usingи без
- Ловушка double → intв действии
- Явный вызов Base::foo

Код лежит в examples/lecture_08_inheritance/05_name_hiding.

Копия в базовый тип теряет производную часть

```cpp
class Base {
public:
        int x = 10;
        virtual ~Base() = default;
        virtual void print() const {
                std::cout << "Base: x=" << x << '\n';
        }
};

class Derived : public Base {
public:
        int y = 20;
        double weight = 1.5;
        void print() const override { /* ... */ }
};

void by_value(Base b) { b.print(); }                    // СРЕЗКА
void by_ref(const Base& b) { b.print(); } // OK

Derived d; Base
b = d;                                  // СРЕЗКА
Base arr[2] = {d, d};                   // СРЕЗКА
by_value(d);                            // СРЕЗКА
```

- копируется только **Base-часть**: поля и переопределения теряются
- срезка происходит в параметрах и контейнерах по значению
- решение — **ссылка** или **указатель**
- sizeof(Base) < sizeof(Derived)— наглядное объяснение

> Срезка не диагностируется компилятором: это штатное копирование базовой части.

ПРИМЕР

Смотрим пример: 06_object_slicing

Разбираем после теории

Срезка объектов

Открыть папку примера →

- sizeofбазы и наследника
- Срезка при копировании, в параметре и в массиве
- Ссылка и указатель сохраняют тип

Код лежит в examples/lecture_08_inheritance/06_object_slicing.

«Реализован посредством» — не is-a

```cpp
// Вариант 1: приватное наследование
class StackPrivate : private std::vector<int> {
public:
    void push(int v) { push_back(v); }void pop()    {
    pop_back(); }int            top() const { return back();
    }bool empty() const {
        return std::vector<int>::empty();
    }
};

// Вариант 2: композиция — предпочтительно
class StackComposed {std::vector<int>
    data_;
public:
    void push(int v) { data_.push_back(v); }void pop()
        { data_.pop_back(); }int                top() const {
    return data_.back(); }
    bool empty() const { return data_.empty(); }
};
```

- оба варианта скрывают интерфейс vector
- приватное наследование оправдано, когда нужен доступ к **protected** или переопределение виртуальных функций
- композиция слабее связывает классы и проще в замене реализации

> Ещё причина: EBO — оптимизация пустого базового класса.

ПРИМЕР

Смотрим пример: 07_private_vs_composition

Разбираем после теории

Приватное наследование vs композиция

Открыть папку примера →

- Два стека: private-наследование и композиция
- Одинаковый сценарий для обоих
- push_backи operator[]наружу не протекают

Код лежит в examples/lecture_08_inheritance/07_private_vs_composition.

Ограничить иерархию и выбрать связь

```
class Framework final {                          // нельзя наследовать
public:
        void run() const { std::cout << "run\n"; }
};
// class My : public Framework {};                // ОШИБКА

class Base {
public:
        virtual ~Base() = default;
        virtual void process() final { /* ... */ }
        virtual void handle() { /* ... */ }
};

// Плохо: машина НЕ является двигателем
class Car : public Engine {};

// Хорошо: машина СОДЕРЖИТ двигатель
class Car {
        Engine engine_;
public:
        std::string start() const { return engine_.ignite(); }
};

// LSP: Circle : Ellipse нарушает контракт
// setRadii(3, 5) превращает круг в не-круг
```

- final на классе запрещает наследование
- final на методе запрещает переопределение
- наследование — только для is-a, проходящего тест **Лисков**
- has-a выражается **композицией**

> Наследование — самая сильная связь между классами; final фиксирует точку, дальше которой она не растёт.

ПРИМЕР

Смотрим пример: 08_final_and_composition

Разбираем после теории

final и «предпочитайте композицию»

Открыть папку примера →

- finaly класса и у метода
- Carсодержит Engineвместо наследования
- LSP-ловушка Circle : Ellipseв комментарии

Код лежит в examples/lecture_08_inheritance/08_final_and_composition.

Две базы — два подобъекта

```
class Printer {
public:
        Printer() { std::cout << "Printer()\n"; }
        void print(const std::string& t) const { /* … */ }
        void status() const { std::cout << "Printer\n"; }
};

class Scanner {
public:
        Scanner() { std::cout << "Scanner()\n"; }std::string scan()
        const { return "doc"; }
        void status() const { std::cout << "Scanner\n"; }
};

class Copier : public Printer, public Scanner {
public:
        Copier() { std::cout << "Copier()\n"; }
        void status() const {                    // разрешаем неоднозначность
                Printer::status();
                Scanner::status();
        }
};

struct A { int a; };   struct B { int b; };
struct C : A, B { int c; };
C obj;          A* pa = &obj;           B* pb = &obj;          // pb смещён
```

- порядок конструкторов — по **списку наследования** слева направо
- одноимённые члены баз требуют **квалификации**
- каждая база — отдельный подобъект со своим смещением
- static_castкорректирует адрес, reinterpret_cast — нет

> Квалификация — не решение проблемы ромба, а способ разрешить конкретное имя.

ПРИМЕР

Смотрим пример: 09_multiple_inheritance

Разбираем после теории

Синтаксис MI, неоднозначность, корректировка указателей

Открыть папку примера →

- Printer+ Scanner⟶ Copier
- Квалификация Printer::status()и Scanner::status()
- Адреса A*и B*внутри C

Код лежит в examples/lecture_08_inheritance/09_multiple_inheritance.

Две копии общего предка

```
class Device {
public:
        int id;
        explicit Device(int id) : id(id) { /* … */ }
};

class Printer : public Device {
public:
        explicit Printer(int id) : Device(id) {}
        void print() const { /* … */ }
};

class Scanner : public Device {
public:
        explicit Scanner(int id) : Device(id) {}std::string scan() const
        { /* … */ }
};

class Copier : public Printer, public Scanner {
public:
        explicit Copier(int id)
                : Printer(id), Scanner(id + 100) {}
};

// c.id;          // ОШИБКА: неоднозначно
// c.Printer::id;                // 1
// c.Scanner::id;                // 101 — независимое состояние
```



- дублируются **данные**, а не только имена
- растёт размер объекта
- изменение «одной» копии не меняет другую
- квалификация масок, а не решает

ПРИМЕР

Смотрим пример: 10_diamond_problem

Разбираем после теории

Проблема ромба

Открыть папку примера →

- Две копии Devicев одном объекте
- Разные адреса id
- sizeof(Copier)больше ожидаемого

Код лежит в examples/lecture_08_inheritance/10_diamond_problem.

virtual гарантирует единственный подобъект

```
class Device {
public:
        int id;
        explicit Device(int id) : id(id) { /* … */ }
};

class Printer : virtual public Device {
public:
        explicit Printer(int id) : Device(id) { /* … */ }
};

class Scanner : virtual public Device {
public:
        explicit Scanner(int id) : Device(id) { /* … */ }
};

class Copier : public Printer, public Scanner {
public:
        // most-derived класс вызывает виртуальную базу
        explicit Copier(int id)
                : Device(id), Printer(id), Scanner(id) {}
};

Copier c(42); c.id;        // Device(42), Printer(), Scanner(), Copier()
                           // OK: одна копия
&c.Printer::id == &c.Scanner::id;                    // true
```

- конструктор виртуальной базы вызывает **most-derived** класс
- вызовы из Printer/Scannerигнорируются
- промежуточные классы хранят vbptr
- цена: объект больше, доступ к базе косвенный

> virtualменяет структуру объекта, а не уровень доступа. Применяйте только при реальной необходимости.

ПРИМЕР

Смотрим пример: 11_virtual_inheritance

Разбираем после теории

Виртуальное наследование

Открыть папку примера →

- Единственный Deviceи порядок конструкторов
- Printerкак most-derived объект
- sizeofи адреса: обычное vs виртуальное наследование

> Код лежит в examples/lecture_08_inheritance/11_virtual_inheritance.

Безопасные формы множественного наследования

```cpp
// Примеси без шаблонов class
Clickable {public:
    void click() const { std::cout << "клик\n"; }
};
class Hoverable {
public:
    void hover() const { std::cout << "наведение\n"; }
};
class Button : public Clickable, public Hoverable {std::string label_;
public:
    explicit Button(std::string l) : label_(std::move(l)) {}
};

// Чистые интерфейсы: классы без данных
class IPrintable {
public:
    virtual void print() const = 0;
    virtual ~IPrintable() = default;
};
class IScannable {
public:
    virtual std::string scan() const = 0;
    virtual ~IScannable() = default;
};
class SmartDevice : public IPrintable, public IScannable {
public:
                void print() const override { /* … */ }
                std::string scan() const override { /* … */ }
 };
```

- примеси безопасны, если нет общей базы и общего состояния
- интерфейсы безопасны, потому что не несут данных
- если класс **владеет** подсистемами — композиция MI
- от классов с данными — почти всегда ошибка

> Если задумались о MI от классов с данными — скорее всего, нужна композиция.

ПРИМЕР

Смотрим пример: 12_mixins_and_alternatives

Разбираем после теории

Примеси, интерфейсы и альтернативы MI

Открыть папку примера →

- A: MI от конкретных классов
- B: композиция
- C: примеси Clickable/Hoverable
- D: чистые интерфейсы

Код лежит в examples/lecture_08_inheritance/12_mixins_and_alternatives.